

剩余知识点题型适配模拟卷

目录

卷面说明	2
一、背景型简答：IR、NFA/DFA、SSA（20 分）	2
二、理论细节：SDD、SDT、属性文法（20 分）	3
三、过程型小题：LR 表格驱动分析器（20 分）	3
四、次级流程题：通用左递归消除（20 分）	5
五、次级流程题：数组类型的 L 属性定义（20 分）	6
六、词法题原型补齐：词素、词法单元与 PPT 原题自动机（20 分）	7
七、LL(1) 原题文法与二进制小数 SDD（20 分）	8
八、AST 构造 SDD 与语法树结点表示（20 分）	10
九、IR 分类、设计维度与 phi 实现策略（20 分）	11
十、控制流翻译：穿越、回填表项与布尔值求值（20 分）	13

卷面说明

本卷只针对前面四套卷没有完全展开的 PPT 内容。题型按知识点适合方式安排。本版采用“题目后紧跟参考答案”的排版，便于复习时对照。

1. 背景型知识点：适合简答、判断、对比。
2. 理论细节：适合解释原因、说明规则。
3. 过程型知识点：适合小计算、小推导。
4. 次级但可出题内容：适合短流程题。

一、背景型简答：IR、NFA/DFA、SSA（20 分）

题目

回答下列问题：

1. 说明高层 IR 和低层 IR 的区别，各举一个特点。
2. 说明为什么 DFA 的执行通常比 NFA 的直接模拟更简单。
3. 说明 SSA 为什么有利于优化。
4. 说明工业编译器中使用 SSA 或类似 SSA 形式的原因。

参考答案

高层 IR 和低层 IR：

高层 IR 保留较多源语言结构，例如循环、函数、对象等，便于前端语义分析和高层优化。
低层 IR 更接近机器，例如显式跳转、加载/存储、寄存器或栈操作，便于后端代码生成。

DFA 比 NFA 更容易直接执行：

DFA 在每个状态、每个输入符号上最多只有一个确定转移。
NFA 可能有多个转移和 `epsilon` 转移，直接模拟时要维护一组可能状态。
因此 DFA 执行时只需要维护一个当前状态。

SSA 有利于优化：

SSA 中每个变量版本只赋值一次，定义和使用关系清楚。
这有利于常量传播、死代码删除、公共子表达式消除等优化。

工业编译器使用 SSA 或类似形式的原因：

SSA 能简化数据流分析，让优化算法更容易实现。
实际编译器在后端会把 `phi` 函数转换成复制、寄存器分配或其它等价低层操作。

二、理论细节：SDD、SDT、属性文法（20 分）

题目

回答下列问题：

1. 说明 SDD 和 SDT 的区别。
2. 说明属性文法和带副作用语义动作的区别。
3. 为什么属性依赖图中存在环时，属性值无法正常求值。
4. 什么是受控副作用，写出两个基本要求。

参考答案

SDD 和 SDT：

SDD 是语法制导定义，偏抽象，描述文法符号有哪些属性以及属性如何计算。

SDT 是语法制导翻译，偏实现，把语义动作嵌入到产生式中，说明分析过程中何时执行动作。

属性文法和带副作用语义动作：

属性文法主要通过属性之间的依赖计算语义信息，不依赖修改全局状态。

带副作用语义动作可能会修改符号表、输出代码或更新全局结构。

依赖图中有环时无法求值：

如果属性 A 依赖属性 B，同时 B 又直接或间接依赖 A，就没有合法的先后计算顺序。

依赖图存在环时，无法得到拓扑序，因此属性值无法正常求出。

受控副作用：

受控副作用是指语义动作虽然会修改外部状态，但执行位置和执行顺序明确，并且不破坏属性求值。

基本要求包括：

1. 副作用执行前所需属性已经计算完成。
2. 副作用不会引入依赖环。
3. 副作用不会因分析顺序不确定而重复执行或漏执行。

三、过程型小题：LR 表格驱动分析器（20 分）

题目

已知某 LR 分析表有如下条目：

```
ACTION[0,id] = s1
ACTION[1,+] = r(E -> id)
ACTION[1,$] = r(E -> id)
ACTION[2,+] = s3
ACTION[2,$] = acc
ACTION[3,id] = s4
ACTION[4,$] = r(E -> E + id)

GOTO[0,E] = 2
```

输入串：

```
id + id $
```

要求：

- 1. 说明 ACTION 表和 GOTO 表分别在什么时候使用。
- 2. 写出分析过程中栈和输入的大致变化。
- 3. 说明什么时候接受输入。

参考答案

ACTION 和 GOTO：

ACTION[state, 终结符] 用于面对输入符号时决定移进、规约或接受。
GOTO[state, 非终结符] 用于规约出非终结符之后决定进入哪个状态。

分析过程：

状态栈	符号栈	输入	动作
0	空	id + id \$	ACTION[0,id] = s1
0 1	id	+ id \$	ACTION[1,+] = r(E -> id)
0 2	E	+ id \$	GOTO[0,E] = 2
0 2 3	E +	id \$	ACTION[2,+] = s3
0 2 3 4	E + id	\$	ACTION[3,id] = s4
0 2 3 4	E + id	\$	ACTION[4,\$] = r(E -> E + id)
0 2	E	\$	ACTION[2,\$] = acc

说明：

接受条件是当前状态和输入结束符 \$ 对应的 ACTION 表项为 acc。

四、次级流程题：通用左递归消除（20 分）

题目

文法：

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow S d \mid c \end{aligned}$$

按非终结符顺序：

$$S, A$$

1. 判断该文法是否存在间接左递归。
2. 按通用左递归消除思路，先把 $A \rightarrow S d$ 中的 S 用 S 的产生式替换。
3. 消除替换后 A 的直接左递归。

参考答案

原文法：

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow S d \mid c \end{aligned}$$

存在间接左递归：

$$S \Rightarrow A a \Rightarrow S d a$$

按顺序 S, A ，处理 A 时，把 $A \rightarrow S d$ 中的 S 替换为 S 的右部：

$$A \rightarrow A a d \mid b d \mid c$$

此时 A 有直接左递归：

$$A \rightarrow A a d \mid b d \mid c$$

消除直接左递归：

$$\begin{aligned} A &\rightarrow b d A' \mid c A' \\ A' &\rightarrow a d A' \mid \epsilon \end{aligned}$$

最终文法：

$$\begin{aligned} S &\rightarrow A a \mid b \\ A &\rightarrow b d A' \mid c A' \\ A' &\rightarrow a d A' \mid \epsilon \end{aligned}$$

五、次级流程题：数组类型的 L 属性定义（20 分）

题目

考虑数组类型声明：

```
T -> B C
B -> int | float
C -> [ num ] C | epsilon
```

设计一个 L 属性 SDD，用于计算类型 `T.type`。要求：

1. `B.type` 表示基本类型。
2. `C.in` 表示从左侧传入的已有类型。
3. `C.type` 表示加上数组维度后的类型。
4. 对声明形式 `int[10][20]` 写出类型构造顺序。

参考答案

语义规则：

```
T -> B C
  C.in = B.type
  T.type = C.type

B -> int
  B.type = int

B -> float
  B.type = float

C -> [ num ] C1
  C1.in = array(num.val, C.in)
  C.type = C1.type

C -> epsilon
  C.type = C.in
```

对 `int[10][20]`：

```
B.type = int
C.in = int

读到 [10]:
C1.in = array(10, int)
```

读到 [20]:

```
C2.in = array(20, array(10, int))
```

C2 -> epsilon:

```
C2.type = array(20, array(10, int))
```

最终:

```
T.type = array(20, array(10, int))
```

说明:

该写法按从左到右的语义动作构造类型。

如果课程约定数组类型按相反嵌套顺序表示, 卷面应按老师 PPT 中的记法调整。

六、词法题原型补齐: 词素、词法单元与 PPT 原题自动机 (20 分)

题目

回答并完成下列小题:

1. 说明词素和词法单元的区别。
2. 对正则表达式 $r = (a|b)^*a^+$, 写出一个 Thompson NFA 的边表。
3. 按子集构造法写出该 NFA 对应 DFA 的主要状态集合和转移。
4. 对密码识别规则写出正则表达式: 字符只含 a,b,0,1, 必须以字母开头, 必须以 abb 结尾, 中间不能出现连续两个数字。

参考答案

词素和词法单元:

词素是源程序中实际出现的字符序列, 例如具体的变量名 `count`。

词法单元是词法分析器输出的类别或记号, 例如 `id`、`num`、`if`。

同一个词法单元可以对应很多不同词素。

$r = (a|b)^*a^+$ 的一种 Thompson NFA 边表:

```
0 --epsilon--> 1
0 --epsilon--> 7
1 --epsilon--> 2
1 --epsilon--> 3
```

```
2 --a--> 4
3 --b--> 5
4 --epsilon--> 6
5 --epsilon--> 6
6 --epsilon--> 1
6 --epsilon--> 7
7 --epsilon--> 8
8 --a--> 9
9 --a--> 9
```

起点: 0
终点: 9

子集构造的主要状态集合:

```
A = epsilon-closure({0}) = {0,1,2,3,7,8}
B = epsilon-closure(move(A,a)) = {1,2,3,4,6,7,8,9}
C = epsilon-closure(move(A,b)) = {1,2,3,5,6,7,8}
```

DFA 转移:

状态	on a	on b	是否终态
A	B	C	否
B	B	C	是
C	B	C	否

密码识别正则表达式:

$(a|b)((a|b)|0(a|b)|1(a|b))^*abb$

说明:

(a|b) 保证以字母开头。
末尾的 abb 保证以 abb 结尾。
中间部分每个数字 0 或 1 后面都必须跟一个字母, 因此不会出现 00、01、10、11。

七、LL(1) 原题文法与二进制小数 SDD (20 分)

题目

给定文法:

```
S -> c S | A B
A -> a A | epsilon
B -> b B a | d
```


1. 计算 FIRST 集和 FOLLOW 集。
2. 构造 LL(1) 预测分析表。
3. 判断该文法是否为 LL(1) 文法。

再给定文法：

```
S -> L . L | L
L -> L B | B
B -> 0 | 1
```

4. 设计 SDD，计算二进制小数对应的十进制值。
5. 用该 SDD 计算 101.101。

参考答案

FIRST 集：

```
FIRST(A) = { a, epsilon }
FIRST(B) = { b, d }
FIRST(S) = { a, b, c, d }
```

FOLLOW 集：

```
FOLLOW(S) = { $ }
FOLLOW(A) = { b, d }
FOLLOW(B) = { a, $ }
```

预测分析表：

```
M[S,c] = S -> c S
M[S,a] = S -> A B
M[S,b] = S -> A B
M[S,d] = S -> A B

M[A,a] = A -> a A
M[A,b] = A -> epsilon
M[A,d] = A -> epsilon

M[B,b] = B -> b B a
M[B,d] = B -> d
```

判断：

每个表项最多只有一条产生式，因此该文法是 LL(1) 文法。

二进制小数 SDD:

属性:

$v(B)$: B 对应的数值

$v(L)$: L 对应的数值

$v(S)$: S 对应的数值

$l(L)$: L 的二进制位数

$S \rightarrow L1 . L2$

$$v(S) = v(L1) + v(L2) / 2^{l(L2)}$$

$S \rightarrow L$

$$v(S) = v(L)$$

$L1 \rightarrow L2 B$

$$v(L1) = 2 * v(L2) + v(B)$$

$$l(L1) = l(L2) + 1$$

$L \rightarrow B$

$$v(L) = v(B)$$

$$l(L) = 1$$

$B \rightarrow 0$

$$v(B) = 0$$

$B \rightarrow 1$

$$v(B) = 1$$

计算 101.101:

左侧 101:

$$v(L1) = 5, l(L1) = 3$$

右侧 101:

$$v(L2) = 5, l(L2) = 3$$

$$v(S) = 5 + 5 / 2^3 = 5 + 0.625 = 5.625$$

八、AST 构造 SDD 与语法树结点表示 (20 分)

题目

给定表达式文法:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

1. 设计构造 AST 的 SDD。
2. 说明 AST 结点对象通常需要保存哪些域。
3. 画出 $a + b * c$ 的 AST。

参考答案

构造 AST 的 SDD:

```
E -> E1 + T
    E.node = Node("+", E1.node, T.node)

E -> T
    E.node = T.node

T -> T1 * F
    T.node = Node("*", T1.node, F.node)

T -> F
    T.node = F.node

F -> ( E )
    F.node = E.node

F -> id
    F.node = Leaf(id.lexeme)
```

AST 结点对象的常见域:

op: 内部结点的运算符或语法构造名称。
children: 指向子结点的指针列表。
value/name: 叶子结点保存的词法值或标识符名字。
type: 语义分析后可附加类型信息。
entry: 标识符结点可绑定符号表条目。

$a + b * c$ 的 AST:

```
      +
     / \
    a  *
     / \
    b  c
```

九、IR 分类、设计维度与 phi 实现策略（20 分）

题目

回答下列问题:

1. 说明高层 IR、中层 IR、低层 IR 的区别。
2. 列出 IR 设计与分析的关键维度。
3. 说明树形 IR、三地址码、SSA、堆栈式 IR 的特点。
4. 说明 phi 函数在后端通常如何展开。

参考答案

高层 IR、中层 IR、低层 IR:

高层 IR 保留函数、循环、异常、对象等源语言结构，适合高层优化。

中层 IR 常见形式是三地址码、四元式、SSA 形式的 TAC，兼顾分析和变换。

低层 IR 接近机器，显式表示寄存器、栈、load/store、分支和调用约定。

IR 设计与分析的关键维度:

控制流表示: 结构化控制流，或显式跳转和基本块图。

数据表示: 三地址临时变量，或堆栈操作数。

赋值模型: 普通赋值，或静态单赋值 SSA。

内存与地址: 显式 load/store，是否暴露别名和指针。

类型与语义信息: 强类型 IR、弱类型 IR，是否保留异常、越界检查、GC 屏障等信息。

可分析性: 是否便于构建 CFG、DFG 和做数据流分析。

常见 IR 范式:

树形 IR: 如 AST，适合前端和早期高层变换，不适合低层优化。

三地址码: 线性、接近机器，便于生成控制流和做数据流分析。

SSA: 每个变量版本只赋值一次，def-use 关系清晰，优化友好。

堆栈式 IR: 操作数隐含在栈上，形式紧凑，但数据依赖不如三地址码显式。

LLVM IR、Sea-of-Nodes 等工业 IR 也属于常见代表。

phi 函数展开:

phi 在 SSA 中表示控制流汇合处按前驱选择不同变量版本。

理想语义可看作并行赋值。

后端通常在每个前驱基本块末尾插入 move，把对应版本复制到汇合点使用的名字。

若边上不能直接插入指令，需要拆分边并增加中间块。

寄存器分配阶段可通过合并不冲突的版本减少复制。

示例:

```
L1:
    x1 = -1
    goto L3
L2:
    x2 = 1
    goto L3
```

L3:

```
x3 = phi(x1 from L1, x2 from L2)
```

```
y = x3 * a
```

展开思路:

L1 末尾插入 $x3 = x1$

L2 末尾插入 $x3 = x2$

L3 中直接使用 $x3$

十、控制流翻译：穿越、回填表项与布尔值求值（20 分）

题目

回答并完成下列小题:

1. 说明 truelist、falselist、nextlist 的含义。
2. 说明 makelist、merge、backpatch、nextquad 的作用。
3. 说明标记非终结符 M 和 N 的作用。
4. 写出 $B1 \ \&\& \ B2$ 、 $B1 \ || \ B2$ 的回填规则。
5. 说明“穿越”和 fall-through 的关系。
6. 将赋值语句 $x = a < b \ \&\& \ c < d$ 翻译为三地址码，要求得到布尔值 0 或 1。

参考答案

列表属性:

B.truelist: B 为真时应跳转、但目标尚未确定的指令位置列表。

B.falselist: B 为假时应跳转、但目标尚未确定的指令位置列表。

S.nextlist: 语句 S 执行完后应跳到下一条语句、但目标尚未确定的指令位置列表。

辅助函数:

makelist(i): 创建只包含指令位置 i 的列表。

merge(p1, p2): 合并两个指令位置列表。

backpatch(p, L): 把列表 p 中所有未填目标的跳转指令目标改为 L。

nextquad: 下一条将要生成的三地址指令编号。

标记非终结符:

M -> epsilon

M.instr = nextquad

N -> epsilon

N.nextlist = makelist(nextquad)

emit("goto _")

说明:

M 用来记录后续代码开始位置, 方便前面的跳转回填到这里。

N 用来生成跳过某段代码的 `goto` 指令, 并把它加入 `nextlist`。

布尔表达式回填规则:

```
B -> B1 && M B2
    backpatch(B1.truelist, M.instr)
    B.truelist = B2.truelist
    B.falselist = merge(B1.falselist, B2.falselist)

B -> B1 || M B2
    backpatch(B1.falselist, M.instr)
    B.truelist = merge(B1.truelist, B2.truelist)
    B.falselist = B2.falselist

B -> E1 relop E2
    B.truelist = makelist(nextquad)
    emit("if E1 relop E2 goto _")
    B.falselist = makelist(nextquad)
    emit("goto _")
```

穿越与 fall-through:

穿越是在语法制导翻译中通过继承属性向下传递未确定跳转目标。

fall-through 表示不生成显式 `goto`, 让控制流自然进入下一条指令。

在优化控制流翻译时, 可以把某些 `true` 或 `false` 目标设为 `fall`, 从而减少冗余 `goto`。

`x = a < b && c < d` 的一种三地址码:

```
if a < b goto L_check2
goto L_false

L_check2:
if c < d goto L_true
goto L_false

L_true:
x = 1
goto L_next

L_false:
x = 0

L_next:
```

说明:

这里布尔表达式不是只改变控制流, 而是要计算赋值右部的逻辑值。

因此最终必须把 `true` 分支落成 `x = 1`, 把 `false` 分支落成 `x = 0`。